Decision Trees

CSCI-P 556

ZORAN TIGANJ

Today

- Decision Trees
- Slides are based on Chapter 6 from the Hands on Machine Learning Textbook.

Decision Trees

- Like SVMs, Decision Trees are versatile Machine Learning algorithms that can perform both classification and regression tasks, and even multioutput tasks.
- Decision Trees are also the fundamental components of Random Forests (next lecture), which are among the most powerful Machine Learning algorithms available today.

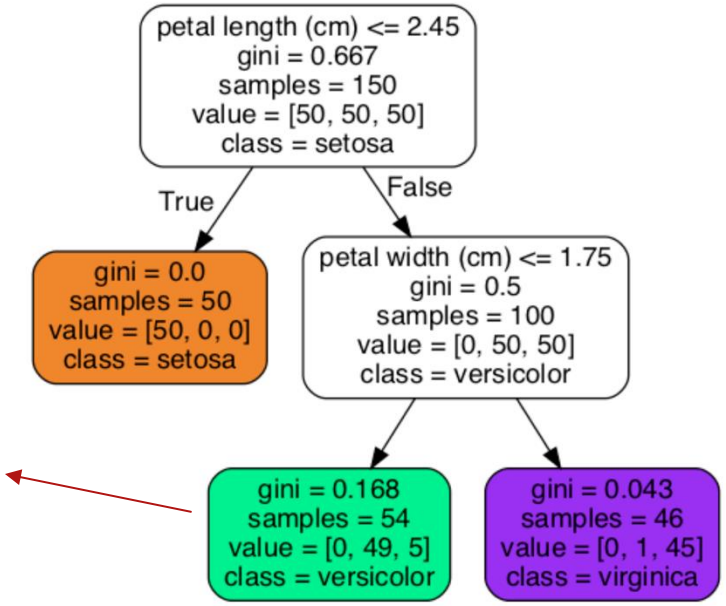
Training and Visualizing a Decision Tree

- To understand Decision Trees, let's build one and take a look at how it makes predictions.
- The following code trains a DecisionTreeClassifier on the iris dataset

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

iris = load_iris()
X = iris.data[:, 2:] # petal length and width
y = iris.target

tree_clf = DecisionTreeClassifier(max_depth=2)
tree_clf.fit(X, y)
```



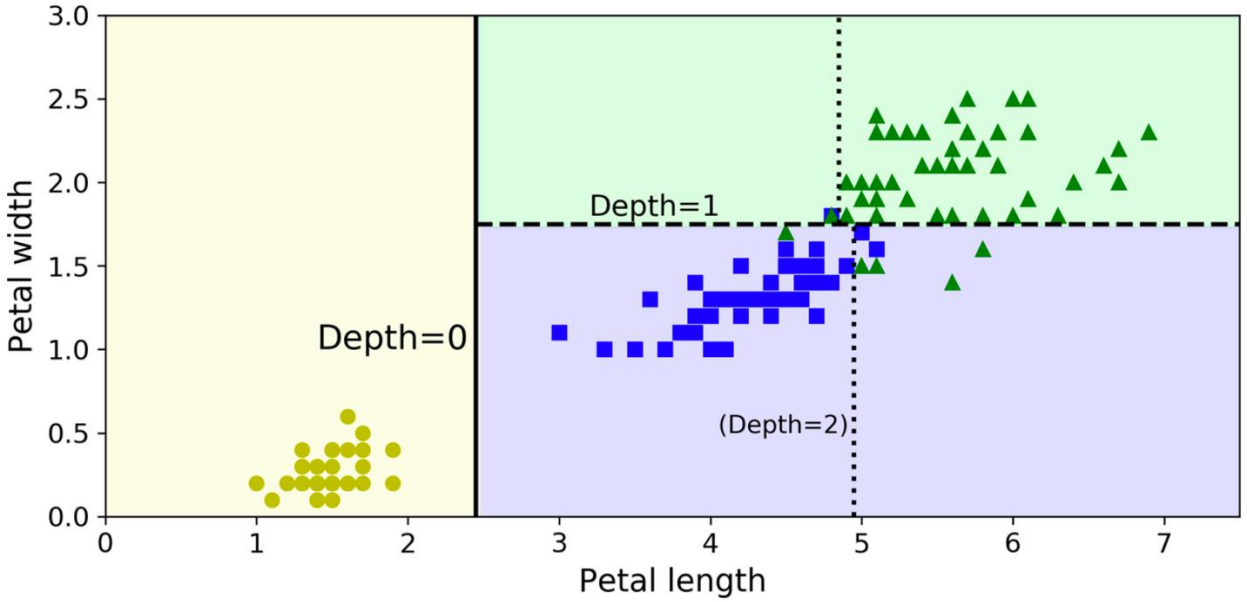
A node's gini attribute measures its impurity: a node is "pure" (gini=0) if all training instances it applies to belong to the same class.

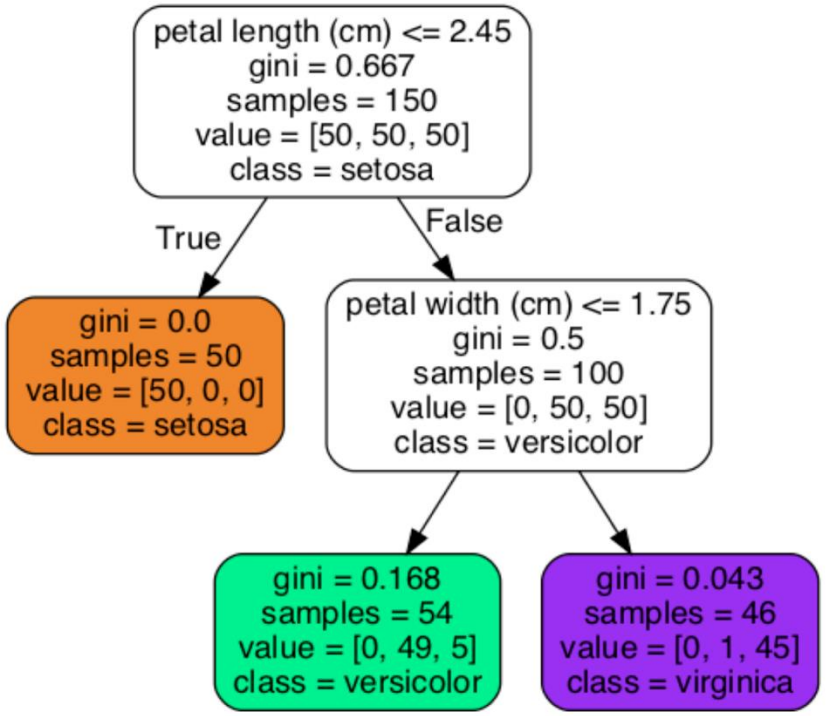
$$1 - (0/54)^2 - (49/54)^2 - (5/54)^2 \approx 0.168$$

$$G_i = 1 - \sum_{k=1}^{n} {p_{i,k}}^2$$

$p_{i,k}$ is the ratio of class k instances among the training instances in the i node.

Training and Visualizing a Decision Tree





Model interpretation

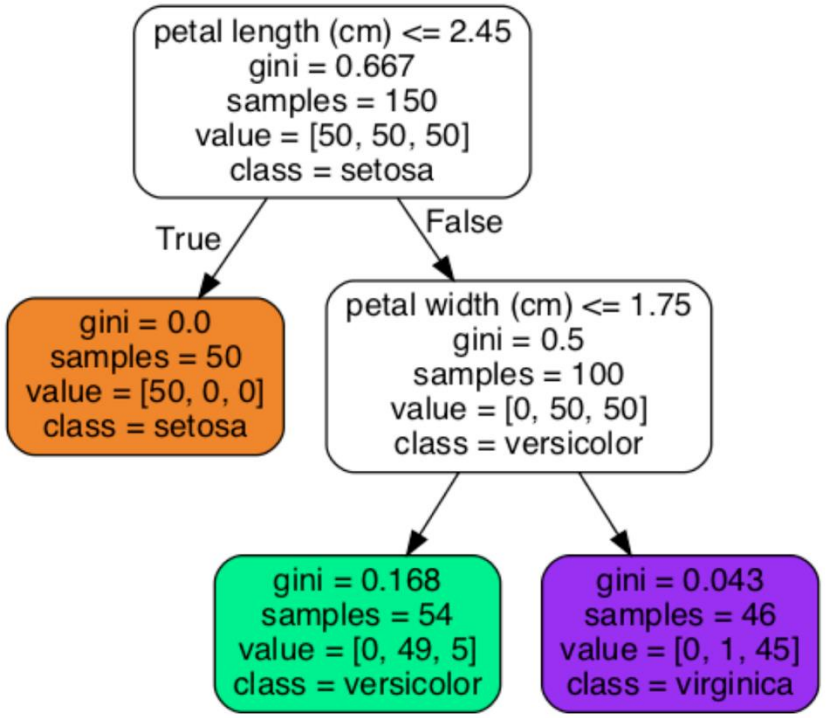
- Decision Trees are intuitive, and their decisions are easy to interpret.
- In contrast, as we will see, neural networks are generally considered **black box models**.
  - For example, if a neural network says that a particular person appears on a picture, it is hard to know what contributed to this prediction: did the model recognize that person's eyes? Their mouth? Their nose? Their shoes? Or even the couch that they were sitting on?
  - Conversely, Decision Trees provide nice, simple classification rules.

Estimating Class Probabilities

- A Decision Tree can also estimate the probability that an instance belongs to a particular class k.
- For example, suppose you have found a flower whose petals are 5 cm long and 1.5 cm wide. The corresponding leaf node is the depth-2 left node, so the Decision Tree should output the following probabilities:
  - 0% for Iris setosa (0/54),
  - 90.7% for Iris versicolor (49/54),
  - 9.3% for Iris virginica (5/54).

```
>>> tree_clf.predict_proba([[5, 1.5]])
array([[0.        , 0.90740741, 0.09259259]])

>>> tree_clf.predict([[5, 1.5]])
array([1])
```



CART training algorithm

- Scikit-Learn uses the Classification and Regression Tree (CART) algorithm to train Decision Trees (also called "growing" trees).
- The algorithm works by first splitting the training set into two subsets using a single feature k and a threshold t_k (e.g., "petal length ≤ 2.45 cm").
- How does it choose k and t_k? It searches for the pair (k, t_k) that produces the purest subsets (weighted by their size).

CART cost function for classification

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$

$$\text{where} \begin{cases} G_{\text{left/right}} \text{ measures the impurity of the left/right subset,} \\ m_{\text{left/right}} \text{ is the number of instances in the left/right subset.} \end{cases}$$

CART training algorithm

- CART algorithm continues splitting subsets until:
  - reaches the maximum depth (defined by the max_depth hyperparameter) or
  - if it cannot find a split that will reduce impurity.

CART training algorithm

- CART algorithm is a **greedy algorithm**: it greedily searches for an optimum split at the top level, then repeats the process at each subsequent level.
- It does not check whether or not the split will lead to the lowest possible impurity several levels down.
- A greedy algorithm often produces a solution that's reasonably good but not guaranteed to be optimal.
- Unfortunately, finding the optimal tree is known to be an NP-Complete problem: it requires O(exp(m)) time, making the problem intractable even for small training sets.

Gini Impurity or Entropy

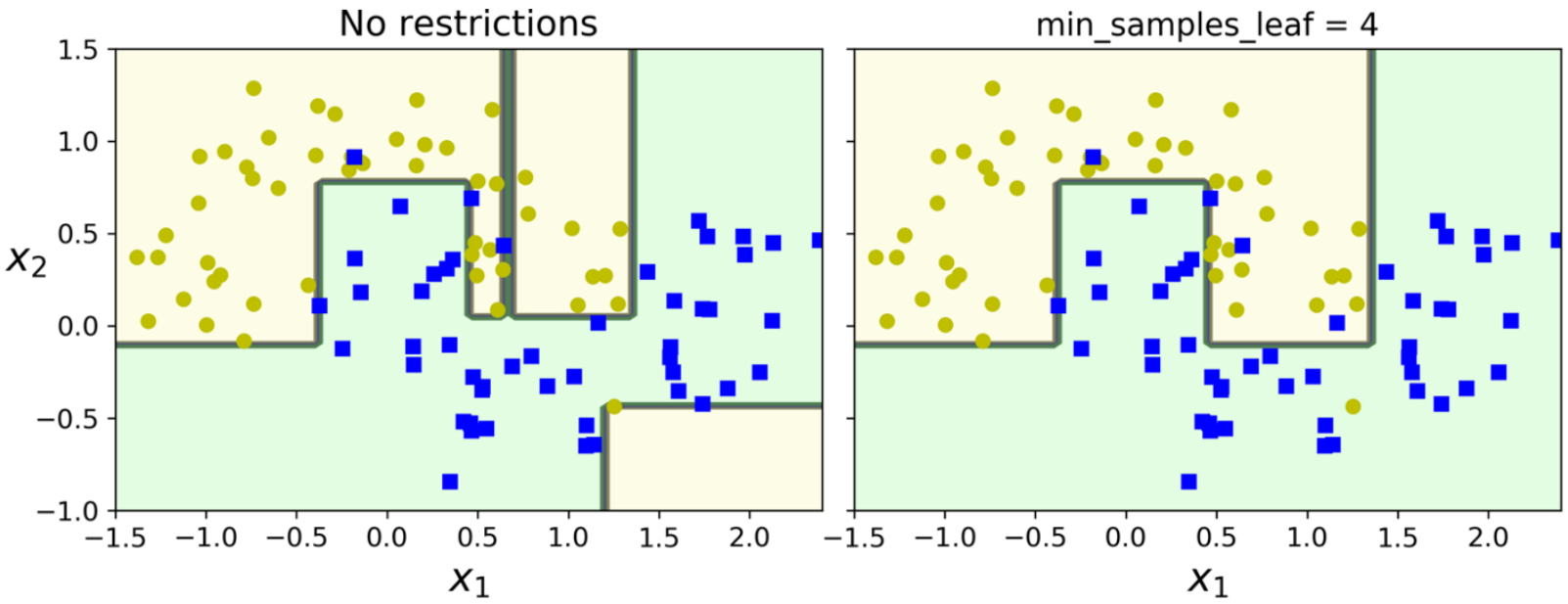
- Alternative to Gini impurity is entropy which measures the average information content of a message: entropy is zero when all messages are identical.
- In Machine Learning, entropy is frequently used as an impurity measure: a set's entropy is zero when it contains instances of only one class.

$$H_i = -\sum_{\substack{k=1 \\ p_{i,k} \neq 0}}^{n} p_{i,k} \log_2 \left(p_{i,k}\right)$$

Regularization Hyperparameters

- In decision trees the number of parameters is not determined prior to training (sometimes called nonparametric model), so the model structure is free to stick closely to the data and if left unconstrained it tends to overfit the data.
- Regularization: To avoid overfitting the training data, you need to restrict the Decision Tree's freedom during training:
- For example, we can limit:
  - maximum depth of the Decision Tree
  - the minimum number of samples a node must have before it can be split
  - the minimum number of samples a leaf node must have

Regularization Hyperparameters

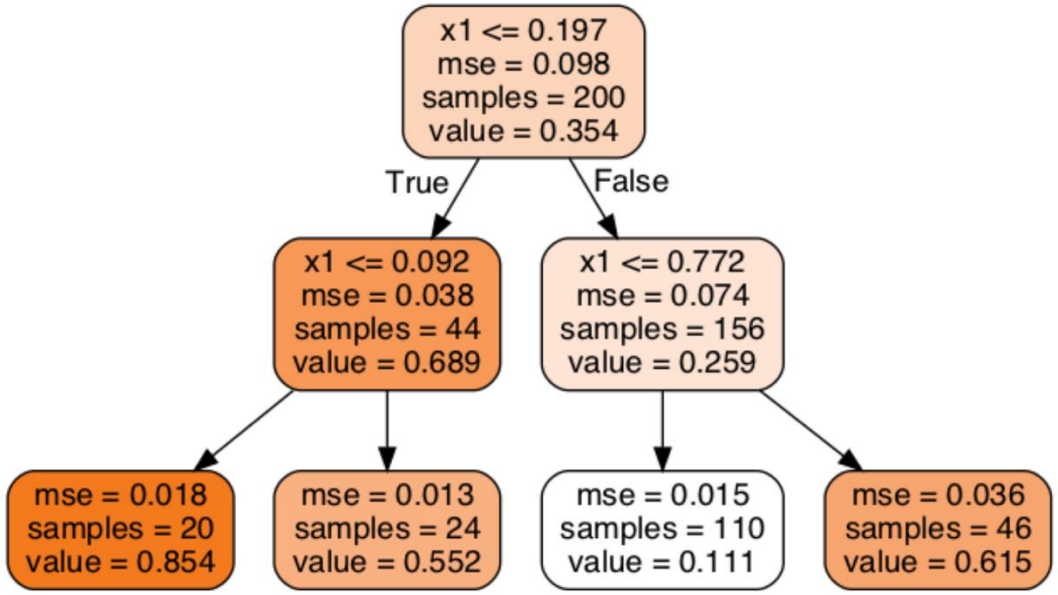


Regression with decision trees

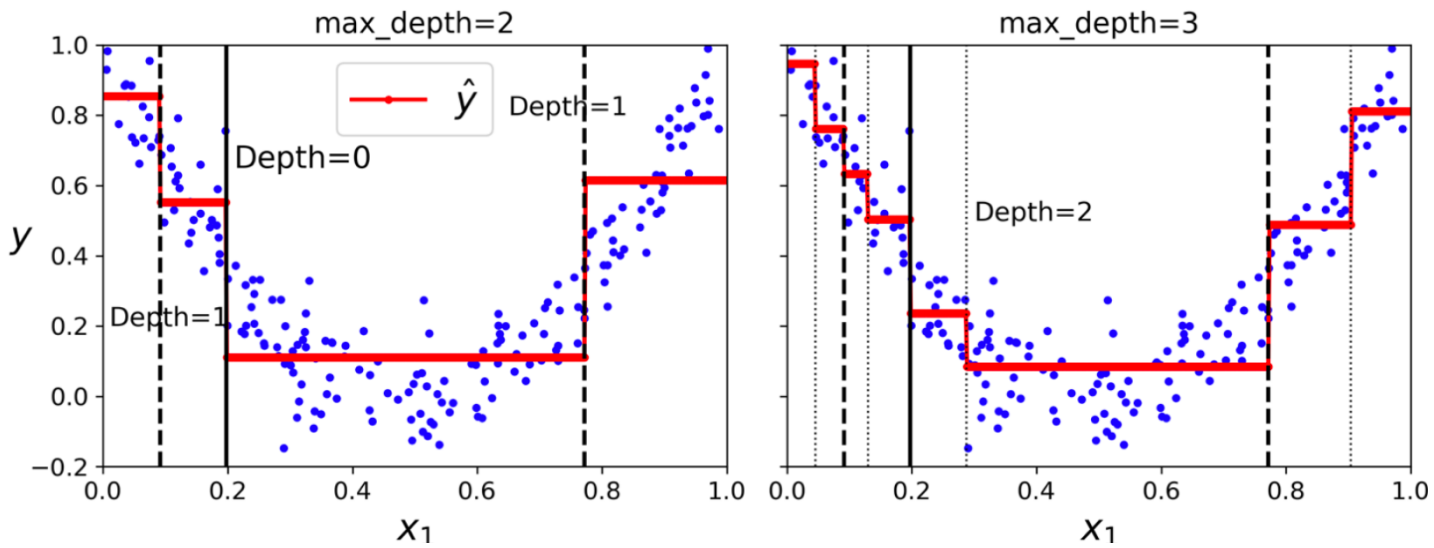
- Training Scikit-Learn's DecisionTreeRegressor class a noisy quadratic dataset

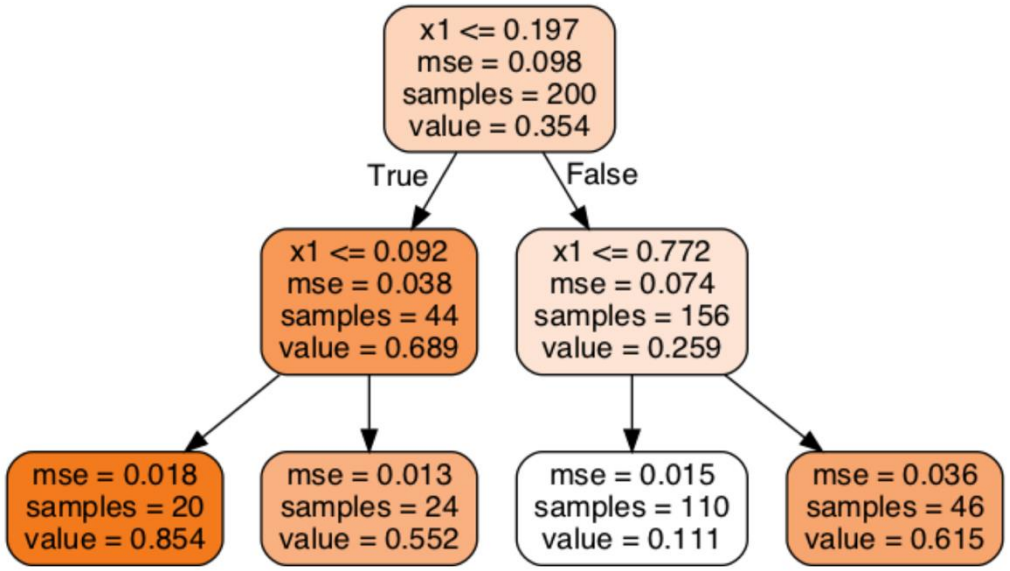
```
from sklearn.tree import DecisionTreeRegressor

tree_reg = DecisionTreeRegressor(max_depth=2)
tree_reg.fit(X, y)
```



Regression with decision trees





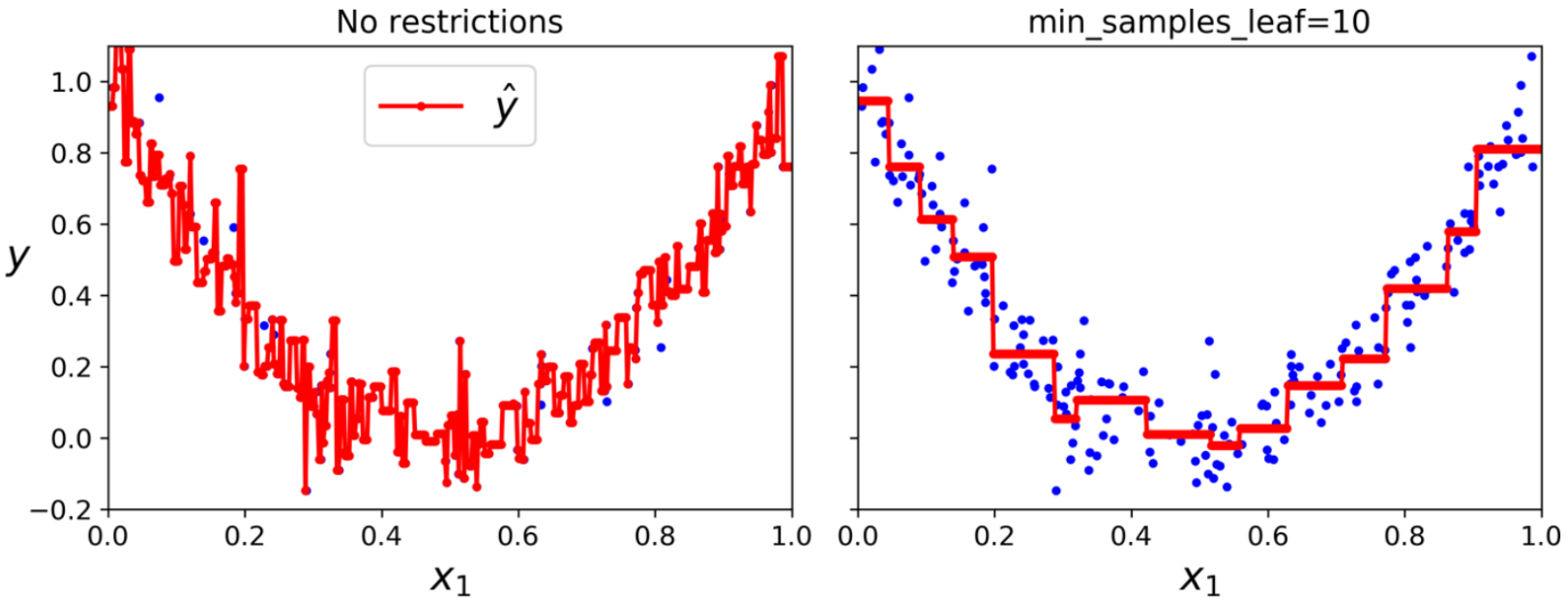
Regression with decision trees

- The CART algorithm works mostly the same way as earlier, except that instead of trying to split the training set in a way that minimizes impurity, it now tries to split the training set in a way that minimizes the MSE.

$$J(k, t_k) = \frac{m_{\text{left}}}{m}\text{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m}\text{MSE}_{\text{right}} \quad \text{where} \begin{cases} \text{MSE}_{\text{node}} = \sum_{i \in \text{node}} \left(\hat{y}_{\text{node}} - y^{(i)}\right)^2 \\ \hat{y}_{\text{node}} = \dfrac{1}{m_{\text{node}}} \sum_{i \in \text{node}} y^{(i)} \end{cases}$$

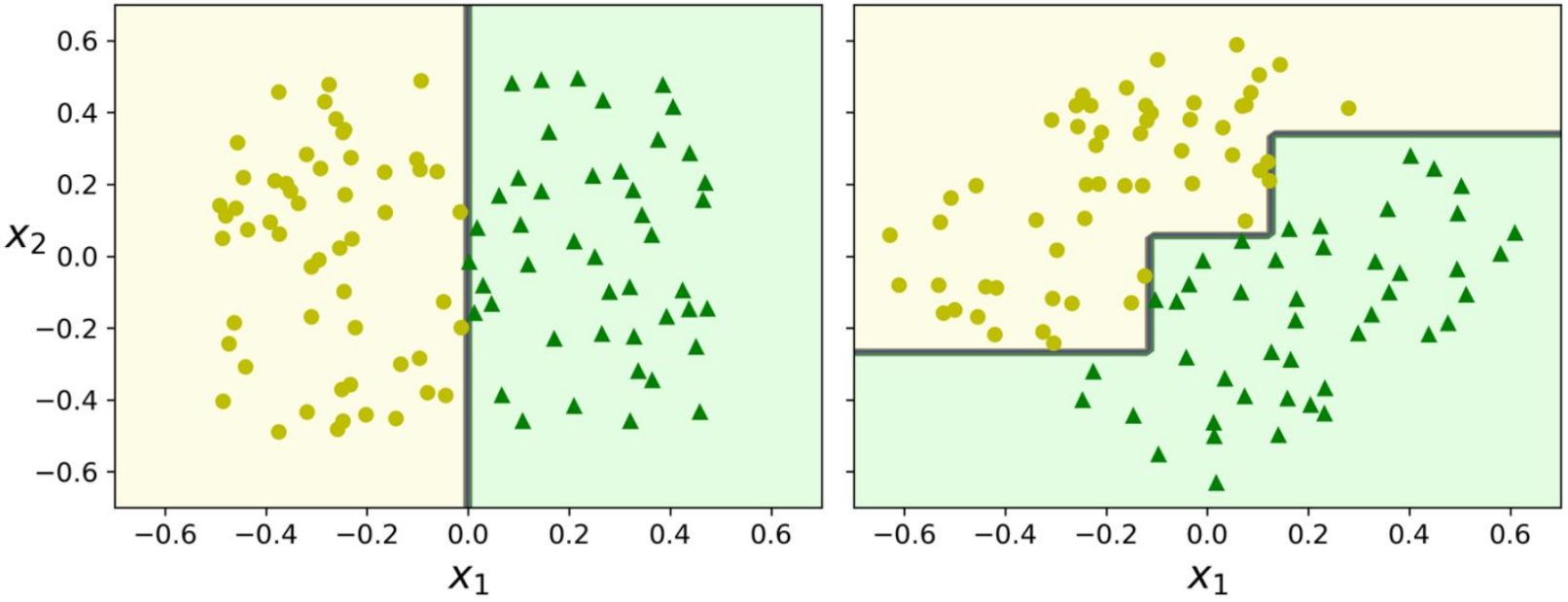
Regression with decision trees

- Just like for classification tasks, Decision Trees are prone to overfitting when dealing with regression tasks so it is important to do regularization:



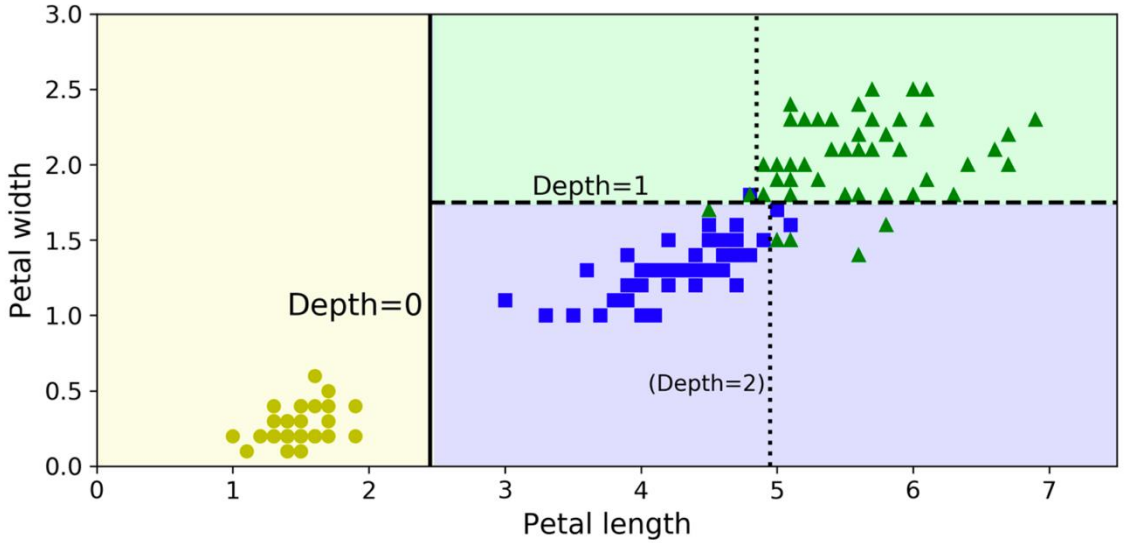
Instability of decision trees

- Decision Trees love orthogonal decision boundaries, which results in sensitivity to training set rotation (figure on the left)



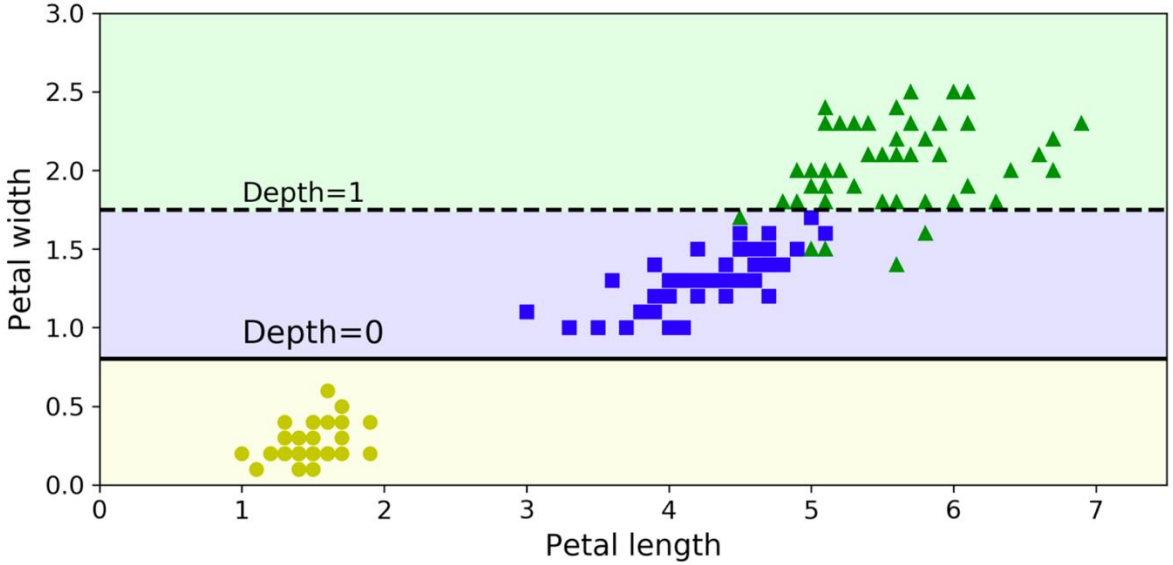
Instability of decision trees

- In general, Decision Trees are that they are very sensitive to small variations in the training data.
  - For example, if we just remove the widest Iris versicolor from the iris training set (the one with petals 4.8 cm long and 1.8 cm wide) and train a new Decision Tree we get a very different results



Instability of decision trees

- In general, Decision Trees are that they are very sensitive to small variations in the training data.
  - For example, if we just remove the widest Iris versicolor from the iris training set (the one with petals 4.8 cm long and 1.8 cm wide) and train a new Decision Tree we get a very different results



Instability of decision trees

- Random Forests can limit this instability by averaging predictions over many trees.

Next time

- Random forests, from Chapter 7, *Hands on machine learning textbook*